

Entrada/Salida

→ Hasta el momento todas las operaciones de entrada/salida se han realizado utilizando los flujos predefinidos **cin** (entrada estándar desde teclado) y **cout** (salida estándar hacia la pantalla), y los operadores de inserción << y extracción >>.

→ Se pueden también leer y escribir ficheros almacenados en el disco duro o en un pendrive.

→ Para realizar operaciones de E/S con archivos se debe incluir en el programa el archivo de cabecera fstream → **#include <fstream>**

→ En C++, antes de abrir un archivo hay que enlazarlo a un flujo (stream).

(1) → Creamos un flujo y declaramos su tipo.

Hay tres tipos de flujos: entrada, salida y entrada/salida.

- **Entrada**: hay que declarar el flujo como de clase ifstream
ifstream fentrada; // se declara el flujo fentrada como de clase ifstream

- **Sálida**: hay que declarar el flujo como de clase ofstream
ofstream fsalida; // se declara el flujo fsalida como de clase ofstream

- **Entrada/salida**: hay que declarar el flujo como de clase fstream
fstream fensal; // se declara el flujo fensal como de clase fstream

(2) → Una vez creado el flujo, hay que asociarlo a un archivo usando la función **open()**.

open(nombrearchivo, modo)

ofstream fsalida;
fsalida.open ("datos.dat", ios::out);

ifstream fentrada;
fentrada.open ("datos.dat", ios::in);
- nombre del archivo: datos.dat
- modo: parámetro opcional que nos indica como se abre el archivo.

ios::in	- apertura para operaciones de entrada (lectura de un fichero) Modo por defecto para flujos de tipo ifstream. Se puede omitir.
ios::out	- apertura para operaciones de salida (grabación de un fichero). Modo por defecto para flujos de tipo ofstream. Se puede omitir.
ios::binary	- apertura en modo binario.
ios::ate	- posicionamiento en el final del archivo cuando éste se abre.
ios::app	- la salida se añade al final de un archivo ya existente (append).
ios::trunc	- se destruye el contenido de un archivo ya existente.

Se pueden combinar varios modos utilizando el operador |.

Por ejemplo para un flujo de entrada/salida:

fstream fensal;
fensal.open("datos.dat", ios::in | ios::out);

→ En la práctica, **la forma más común** de abrir un archivo es con una única sentencia que nos permite crear un flujo declarando su clase, y asociar dicho flujo con un archivo en un sólo paso:

```
ofstream fsalida("datos.dat"); // abre el archivo datos.dat para salida
```

```
ifstream fentrada("datos.dat"); // abre el archivo datos.dat para entrada
```

→ Para chequear si un flujo se ha abierto correctamente se puede utilizar el propio nombre del flujo, ya que este toma el valor 0 si algo falla (hay otras formas...)

```
if (!fentrada)
{
    cout << "No se puede abrir el archivo";
    return 1;
}
```

→ Una vez que el archivo está abierto con un flujo asociado al mismo, podemos tener acceso a dicho archivo utilizando los operadores de inserción << y extracción >> de forma similar que con los flujos estándar cin y cout.

```
fsalida << a << " " << b << endl; // graba a y b en el fichero datos.dat
```

```
fentrada >> a >> b; // lee a y b del fichero datos.dat
```

→ Una vez realizadas todas las operaciones de lectura o escritura de datos es necesario cerrar el archivo utilizando la función **close()** para cerrar el flujo asociado al archivo.

```
fsalida.close();
```

```
fentrada.close();
```

→ Se puede detectar cuando se ha alcanzado el final del archivo usando la función **eof()**, ya que esta función devuelve un valor distinto de cero tan sólo cuando se ha alcanzado el final del archivo.

```
.....
while (!fentrada.eof())
{
    fentrada >> data[i];
    i=i+1;
}
.....
```

→ Las funciones **open()**, **close()**, **eof()** son específicas de objetos de tipo “flujo”.

Ejemplo:

```
// ejemplo de lectura y escritura de ficheros: se generan varios arrays
// unidimensionales y luego se graban, para posteriormente leerlos
#include <iostream>
#include <fstream>
#include <cmath>
using namespace std;
int main()
{
    int m=100, i;
    char nombrearchivo[30];
    double x[m], y[m], z[m], a[m], b[m], c[m], d[m];
    for (i=0; i<m; i++){
        x[i]= pow(i+1,1.0);
        y[i]= pow(i+1,2.0);
        z[i] = pow(i+1,3.0);
    }
    cout << "Introduzca nombre del archivo: ";
    cin >> nombrearchivo;
    // Crear un flujo de salida y llamarlo fsalida
    // ofstream fsalida;
    // fsalida.open(nombrearchivo, ios::out);
    ofstream fsalida (nombrearchivo, ios::out); // lo hacemos en una sola linea
    if (!fsalida)
    {cout << "No se puede abrir el archivo";
      return 1; }
    for (i=0; i<m; i++){
        fsalida << i+1 << "    " << x[i] << "    " << y[i] << "    " << z[i] << endl;
    }
    fsalida.close(); // cerrar el archivo
    // Crear un flujo de entrada y llamarlo fentrada
    // Leemos el archivo que acabamos de grabar
    // ifstream fentrada;
    // fentrada.open(nombrearchivo, ios::in);
    ifstream fentrada(nombrearchivo, ios::in); // lo hacemos en una sola linea
    if (!fentrada)
    { cout << "No se puede abrir el archivo";
      return 1; }
    for (i=0; i<m; i++){
        fentrada >> a[i] >> b[i] >> c[i] >> d[i];
        cout << a[i] << "    " << b[i] << "    " << c[i] << "    " << d[i] << endl;
    }
    fentrada.close(); // cerrar el archivo
    return 0;
}
```

Ejemplo:

```
// ejemplo de lectura y escritura de ficheros: lectura de un fichero de datos
// con dos columnas, x e y, que luego grabamos con otro nombre.
// El nombre del archivo y el numero de puntos se introducen por teclado
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int m,i;
    char nombrearchivo[30], nombrenuevo[30];
    cout << "Introduzca nombre del archivo: ";
    cin >> nombrearchivo;
    cout << "Introduzca numero de puntos: ";
    cin >> m;
    double datax[m], datay[m];
    // Crear un flujo de entrada y llamarlo fentrada
    ifstream fentrada(nombrearchivo, ios::in);
    if (!fentrada)
    { cout << "No se puede abrir el archivo";
      return 1; }
    for (i=0; i<m; i++){
        fentrada >> datax[i] >> datay[i];
        cout << i+1 << " " << datax[i] << " " << datay[i] << endl;
    }
    fentrada.close(); // cerrar el archivo
    cout << "Introduzca nombre nuevo del archivo: ";
    cin >> nombrenuevo;
    // Crear un flujo de salida y llamarlo fsalida
    ofstream fsalida (nombrenuevo, ios::out);
    if (!fsalida)
    { cout << "No se puede abrir el archivo";
      return 1; }
    for (i=0; i<m; i++){
        fsalida << datax[i] << " " << datay[i] << endl;
    }
    fsalida.close(); // cerrar el archivo
    return 0;
}
```

Ejemplo:

```
// ejemplo de lectura y escritura de ficheros: lectura de un fichero de datos
// con dos columnas, x e y, que luego grabamos con otro nombre.
// El numero de puntos del archivo, m, se obtiene utilizando eof
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int m=0, i;
    char nombreamchivo[30], nombrenuevo[30];
    double datax[1000], datay[1000]; // dimensionamos estos arrays en exceso
    cout << "Introduzca nombre del archivo: ";
    cin >> nombreamchivo;
    // Crear un flujo de entrada y llamarlo fentrada
    ifstream fentrada(nombreamchivo, ios::in);
    if (!fentrada)
        { cout << "No se puede abrir el archivo";
          return 1; }
    // utilizacion de eof
    while (!fentrada.eof())
        {
            fentrada >> datax[m] >> datay[m];
            cout << m+1 << " " << datax[m] << " " << datay[m] << endl;
            m=m+1;
        }
    fentrada.close(); // cerrar el archivo
    cout << "Introduzca nombre nuevo del archivo: ";
    cin >> nombrenuevo;
    // Crear un flujo de salida y llamarlo fsalida
    ofstream fsalida (nombrenuevo, ios::out);
    if (!fsalida)
        { cout << "No se puede abrir el archivo";
          return 1;}
    for (i=0; i<m; i++){
        fsalida << datax[i] << " " << datay[i] << endl;
    }
    fsalida.close(); // cerrar el archivo
    return 0;
}
```

Ejemplo:

```
// Ejemplo de lectura y escritura de ficheros: lectura de un fichero de datos
// con dos columnas, x e y, que luego grabamos con otro nombre.
// El numero de puntos del archivo, m, se obtiene utilizando eof. No se
// dimensiona en exceso ningun array. Solo se dimensionan una vez conocido m.
// No usamos funciones. Se utilizan dos flujos de entrada (fentrada y fentrada2)
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int m=0, i;
    char nombreadarchivo[30], nombrenuevo[30];
    double data;
    cout << "Introduzca nombre del archivo: ";
    cin >> nombreadarchivo;
    cout << nombreadarchivo << endl;
    // Crear un flujo de entrada y llamarlo fentrada
    ifstream fentrada(nombreadarchivo, ios::in);
    if (!fentrada) {
        cout << "No se puede abrir el archivo";
        return 1; }
    while (!fentrada.eof()) // utilizacion de eof
    {
        fentrada >> data;
        fentrada >> data;
        m=m+1;
    }
    cout << m << " " << "puntos" << endl;
    fentrada.close(); // cerrar el archivo
    double datax[m], datay[m]; // dimensionamos datax y datay una vez conocido m
    ifstream fentrada2(nombreadarchivo, ios::in);
    if (!fentrada2) {
        cout << "No se puede abrir el archivo";
        return 1; }
    for (int i=0; i<=m-1; i++)
    {
        fentrada2 >> datax[i] >> datay[i];
        cout << i+1 << " " << datax[i] << " " << datay[i] << endl;
    }
    fentrada2.close(); // cerrar el archivo
    cout << "Introduzca nombre nuevo del archivo: ";
    cin >> nombrenuevo;
    cout << nombrenuevo << endl;
    // Crear un flujo de salida y llamarlo fsalida
    ofstream fsalida (nombrenuevo, ios::out);
    if (!fsalida) {
        cout << "No se puede abrir el archivo";
        return 1; }
    for (i=0; i<m; i++){
        fsalida << datax[i] << " " << datay[i] << endl;
    }
    fsalida.close(); // cerrar el archivo
    return 0;
}
```

Ejemplo:

```
// ejemplo de lectura y escritura de ficheros: lectura de un fichero de datos
// con dos columnas, x e y, que luego grabamos con otro nombre. No se dimensiona
// en exceso ningun array. Solo se dimensionan una vez conocido m.
// En este ejemplo a diferencia del anterior utilizamos funciones para obtener
// el numero de puntos m del archivo utilizando eof, y abrir y cerrar archivos.
```

```
#include <iostream>
#include <fstream>
using namespace std;
int numerodepuntos(char nombrearchivo[]);
void leerdatos(char nombrearchivo[], double x[], double y[], int m);
void grabardatos(char nombrearchivo[], double x[], double y[], int m);

int main()
{
    int m;
    char nombrearchivo[30], nombrenuevo[30];
    cout << "Introduzca nombre del archivo: ";
    cin >> nombrearchivo;
    cout << nombrearchivo << endl;

    // Obtencion del numero de puntos m con la funcion numerodepuntos
    m=numerodepuntos(nombrearchivo);
    cout << m << "puntos" << endl;
    if (m==-1) {
        cout << "ERROR. No se puede leer el archivo." << endl;
        return 1;
    }

    double datax[m], datay[m]; // dimensionamos datax y datay una vez conocido m

    // leemos los datos del fichero
    leerdatos(nombrearchivo, datax, datay, m);
    cout << "Introduzca nombre nuevo del archivo: ";
    cin >> nombrenuevo;
    cout << nombrenuevo << endl;

    // grabamos los datos en ficheronuevo
    grabardatos(nombrenuevo, datax, datay, m);
    return 0;
}

int numerodepuntos(char nombrearchivo[])
{
    int m=0;
    double x, y;
    // Crear un flujo de entrada y llamarlo fentrada
    ifstream fentrada(nombrearchivo, ios::in);
    if (!fentrada) { return -1; }
    while (!fentrada.eof()) // utilizacion de eof
    {
        fentrada >> x >> y;
        m=m+1;
    }
    fentrada.close(); // cerrar el archivo
    return m;
}
```

```

void leerdatos(char nombreadarchivo[], double x[], double y[], int m)
{
    // Crear un flujo de entrada y llamarlo fentrada
    ifstream fentrada(nombreadarchivo, ios::in);
    for (int i=0;i<=m-1;i++)
    {
        fentrada >> x[i] >> y[i];
        cout << i+1 << " " << x[i] << " " << y[i] << endl;
    }
    fentrada.close(); // cerrar el archivo
}

void grabardatos(char nombreadarchivo[], double x[], double y[], int m)
{
    // Crear un flujo de salida y llamarlo fsalida
    ofstream fsalida (nombreadarchivo, ios::out);
    for (int i=0; i<m; i++){
        fsalida << x[i] << " " << y[i] << endl;
    }
    fsalida.close(); // cerrar el archivo
}

```

E/S con formato.

→ Hay dos formas de dar formato a la salida: (1) utilizando funciones específicas de objetos de tipo “flujo”, y (2) utilizando manipuladores de flujo.

→ (1) - Utilización de funciones específicas de objetos de tipo “flujo”.

- Se utilizan las funciones **setf()** y **unsetf()** para activar y desactivar, respectivamente, los indicadores de formato que controlan la forma en que se formatea la información.

Indicadores de formato más utilizados:

left	salida justificada a la izquierda
right	salida justificada a la derecha
showbase	se muestra la base de los valores numéricos
scientific	los números en coma flotante se muestran en notación científica
showpos	se muestra el signo + antes de los valores positivos

Por ejemplo para el flujo estándar cout:

cout.setf(ios::showpos); // activa el indicador showpos

cout.setf(ios::scientific); // activa el indicador scientific

cout.setf(ios::showpos | ios::scientific); // combinando varios indicadores con el operador |.

cout.unsetf(ios::showpos | ios::scientific); // desactiva los indicadores showpos y scientific.

- Así mismo tenemos también las funciones:

- **width()** : **cout.width(10)** // se utiliza un campo de 10 caracteres

- **fill()** : **cout.fill('#');** // carácter de relleno #

- **precision()** : **cout.precision(2);** dos dígitos después del punto decimal

Ejemplo:

```
// Ejemplo de utilización de setf(), width(), fill() y precision().
#include <iostream>
using namespace std;
int main()
{
    cout.setf(ios::showpos);
    cout.setf(ios::scientific);
    cout << 123 << " " << 123.23 << "\n";
    cout.precision(2); // dos dígitos después del punto decimal
    cout.width(10);    // se utiliza un campo de 10 caracteres
    cout << 123 << " " << 123.23 << "\n";
    cout.fill('#');    // caracter de relleno #
    cout.width(10);    // se utiliza un campo de 10 caracteres
    cout << 123 << " " << 123.23 << endl;
    return 0;
}
```

The program displays this output:

```
+123 +1.232300e+002
      +123 +1.23e+002
#####+123 +1.23e+002
```

→ (2) Utilización de manipuladores de flujo.

→ Para utilizar las funciones especiales llamadas manipuladores de flujo se debe incluir en el programa el archivo de cabecera iomanip → **#include <iomanip>**

Manipuladores de flujo más utilizados:

setw()	establece la anchura de campo (E/S) → cout << setw(10) << 123.23;
setprecision()	establece el número de dígitos de precisión (E/S) → cout << setprecision(5) << 1000.243;
ws	ignora los espacios blancos iniciales (E)
endl	salida de un carácter de nueva línea y vaciado del flujo (S)
setfill()	establecer el carácter de relleno (E/S) → cout << setfill('#') << 123.23;
setiosflags()	activa los indicadores especificados en su argumento (E/S) ios::scientific - los números en coma flotante se muestran en notación científica → cout << setiosflags(ios::scientific); ios::showbase - se muestra la base de los valores numéricos → cout << setiosflags(ios::showbase); ios::showpos - se muestra el signo + antes de los valores positivos → cout << setiosflags(ios::showpos);
resetiosflags()	desactiva los indicadores especificados en su argumento (E/S)

Ejemplo:

```
//Uso de manipuladores de flujo.
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    cout << setprecision(5) << 1000.243 << endl;
    cout << setw(20) << "Hello there.";
    return 0;
}
```

Salida del programa:

```
1000.2
      Hello there.
```

Ejemplo:

```
// Uso de setiosflags().
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    cout << setiosflags(ios::showpos) << setiosflags(ios::scientific);
    cout << 123 << " " << 123.23 << endl;
    return 0;
}
```

Salida del programa:

```
+123 +1.232300e+002
```

Ejemplo:

```
#include <iostream>
#include <iomanip.h>
using namespace std;
int main()
{
    const int n =15;
    float dato = 123.4567;
    cout << n << endl;           // salida en base diez
    cout << oct << n << endl;     // salida en base octal
    cout << hex<< n << endl;     // salida en base hexadecimal
    cout <<setw(8) << " hola" << endl; // anchura de campo 8
    cout << setw(10); // anchura de campo 10
    cout.fill('#'); // rellena con #
    cout << 34 << endl; // escribe 34 en base 16 anchura 10 y relleno
    // precision 5, 3 dígitos antes del punto decimal y 2 dígitos después
    cout<< setprecision(5) << dato << endl;
    return 0;
}
```

resultado de ejecución



```
15
17
f
  hola
#####22
123.46
Presione una tecla para continuar . .
```